DEPENDENCY-VALIDATED DELIVERY OF LOCAL LEARNING UPDATES

Technical field

[0001] Distributed and event-local learning runtimes with independently updated parameter banks.

Background

[0002] A delayed credit packet can become invalid when an upstream parameter or memory changes even though its receiving bank has not changed. A target-bank version alone does not establish current-context validity.

Summary

[0003] A runtime associates credit with a complete declared dependency set, joins required producers, and validates the relevant parameter and state stamps before an exactly-once local application.

Brief description of the drawings

[0004] FIG. 1 is a block diagram of an implementation. FIG. 2 shows operations of the implementation. Reference signs identify the same functional elements throughout the drawings.

Detailed description

[0005] In FIG. 1, 100 identifies credit packets; 110 identifies bank owner; 120 identifies dependencies; 130 identifies validator; 140 identifies optimizer; 150 identifies snapshots. In FIG. 2, 210 identifies producer join, 220 identifies version check and 230 identifies commit once.

[0006] A bank owner 110 defines a canonical parameter schema, optimizer state and bank version. A teacher packet 120 identifies the target event, computation lineage, receiving bank and all parameter/state versions read by its declared credit target. Factual and counterfactual targets may require different dependency sets.

[0007] For target $F_e(\theta_D, s_S)$, the packet contains stamps $\{(j, v_j) : j \in D\}$ and $\{(k, u_k) : k \in S\}$. Validator 130 compares all declared stamps under a serialized commit boundary. Changes outside D, S do not invalidate this declared computation. Completeness of the dependency set is a separate requirement; small packet size does not establish it.

[0008] A bank names its producer-completion set. Fragments are checked for event, schema and issued version and aggregated in canonical order. A shared parameter leaf has one owner. The bank applies an update only after all required fragments have arrived and marks the target/version token consumed, preventing a replay from applying the same update twice.

[0009] Immutable issued storage can serve outstanding actor reads while complete banks update a live learner. Snapshot references retain the data they address until all dependent reads complete. A local norm cap can be calibrated under a declared aggregate budget, with calibration work recorded separately. A local cap does not claim equality to a different global clipping rule.

[0010] In a genuinely concurrent circuit, validation and commit require a coherence protocol, dependency leases or commit-time revalidation that prevents a dependency changing between check and application. A serialized software validation is a reference embodiment, not a complete distributed atomicity protocol.

[0011] FIG. 1 shows packets 100, bank owner 110, dependency stamps 120, validator 130, optimizer 140 and issued snapshots 150. FIG. 2 shows producer join 210, stamp validation 220 and exactly-once commit 230. A hardware embodiment banks memories and moments locally and carries compact identifiers over a ready/valid or request/acknowledge channel. Dependency bytes, joins, retries and coherence traffic are included in measurements.

Hardware implementation

[0012] A digital implementation may use synthesizable register-transfer logic mapped to an FPGA or to an application-specific integrated circuit. Addressed state uses register banks or SRAM macros; arithmetic uses integer, fixed-point or floating-point datapaths. Ready/valid or request/acknowledge interfaces apply backpressure without accepting a message twice. Synchronous FPGA execution, locally clocked circuits and asynchronous circuits are alternative implementations, each preserving its declared causal ordering and numerical precision.

[0013] Before deployment, a reference trace can check selected addresses, values, timestamps, publication versions and output probabilities against the circuit at the declared numerical precision. Timing quantization, ties, cancellation, overflow and saturation are specified for the implementation. FPGA resource utilization, post-layout timing and measured energy are implementation-dependent quantities; no particular measured hardware result is required by this embodiment.

CLAIMS

1. A method of delivering learning credit in an event-driven processing system, comprising assigning parameters and optimizer state to a canonical receiving bank; receiving a credit packet identifying an event target and a declared parameter and memory dependency set; joining a declared set of producer fragments for the target; validating versions of the declared dependencies at an application boundary; and applying a bank update exactly once when the producer and dependency conditions hold.

2. The method of claim 1, wherein a change to a declared upstream parameter invalidates a packet even when the receiving bank version is unchanged.

3. The method of claim 1, wherein a change outside the declared dependency set does not invalidate the packet.

4. The method of claim 1, wherein immutable issued storage serves outstanding actor reads while a completed receiving bank updates live learner state.

5. The method of claim 1, wherein concurrent validation uses a lease or commit-time revalidation to protect the validation-to-application interval.

6. The method of claim 1, wherein a replayed consumed target token is rejected.

7. A learning runtime apparatus comprising producer inboxes, dependency-version storage, a validator and bank-local optimizer storage configured to perform the method of claim 1.

8. A non-transitory computer-readable storage medium storing instructions that cause a processor to perform the method of claim 1.

ABSTRACT

A learning runtime assigns parameters and optimizer state to canonical banks. A credit packet names an event target and its declared parameter and memory dependencies. The runtime joins required producer fragments, validates dependency versions at an application boundary and applies each accepted target once. Outstanding reads may use immutable issued storage while completed banks update a live learner. Concurrent implementations protect validation through commit. The system records dependency, join and retry traffic separately from gradient payloads.